

Unit-4: Packages, Exception Handling & I/O

A package is a pack(group) of classes, interfaces and other packages. In java we use packages to organize our classes and interfaces and treat it like a single unit.

In Java applications, packages are able to provide the following advantages.

- Reusability

When working on a Java project, you might notice that you keep writing the same code repeatedly. To avoid this, you can create reusable classes and put them in packages. This way, whenever you need to perform that task again, you just import the package and use the class

- Modularity/better organization

Again, in large java projects where we have several hundreds of classes, it is always required to group the similar types of classes in a meaningful package name so that you can organize your project better and when you need something you can quickly locate it and use it, which improves the efficiency.

- **Name Conflicts:** We can define two classes with the same name in different packages so to avoid name collision, we can use packages
- Abstraction

If we declare classes and interfaces in a module or in a package then that classes and interfaces are not visible in outside of the package by default, so that, packages are able to improve Abstraction.

- Security

In Java applications, packages are able to provide Abstraction and Encapsulation, where both are able to improve Security.

Types of packages in Java

There are 2 types of packages in Java .

• Predefined Packages or builtin packages-

These packages are provided by JAVA programming language along with Java software.

Ex: java.lang

java.io

java.util

java.sql

java.net

In java we have several built-in packages, for example when we need user input, we import a package like this:

```
import java.util.Scanner
```

Here:

→ **java** is a top level package

→ **util** is a sub package

→ and **Scanner** is a class which is present in the sub package **util**.

• User defined Packages

These packages are defined by the developers as per their application requirements.

If we declare classes and interfaces in a package and if we want to use these classes and interfaces in the present java file then we have to import the respective package to the present java file.

To import packages to the present java file we have to use the following syntaxes.

```
import package_Name.*;
```

--> It able to import all the classes and interfaces of the specified package.

EX: import java.util.*;

```
import package_Name.member_Name;
```

It able to import only the specified member from the specified package.

EX: import java.util.ArrayList;

Note: In java files, we are able to provide at most one package declaration statement, but, we are able to provide any number of import statements.

User defined Packages:

To **create a class inside a package, declare the package name as the first statement in your program**. A class can have only one package declaration.

Create a class Happy inside a package mypack(Happy.java)

```
package mypack;

public class Happy {

    public static void main(String[] args){

        Happy m = new Happy();

        m.display();

    }

    public void display() {

        System.out.println("This is my package-mypack-Happy!");

    }

}

// first compile this program by javac -d . Happy.java
// to run it java mypack.Happy
```

To use this package in another program PackageDemo.java (import the package)

```
import mypack.Happy;

public class PackageDemo {

    public static void main(String args[]) {

        Happy m = new Happy();

        m.display();

    }

}
```

Path Environment Variable:

The Path environment variable in an operating system specifies the directories in which the system looks for executable files. **It allows you to run programs from the command line without specifying their full path.**

When you type a command in the terminal or command prompt, the system searches through the directories listed in the Path variable to find the executable.

For example, **if javac (Java compiler) is in the Path, you can compile Java programs from any directory without navigating to the JDK bin folder.**

You have the Java Development Kit (JDK) installed in the directory C:\Program Files\Java\jdk-17\bin.

Without Setting the Path:

Every time you want to compile a Java program, you would have to navigate to the JDK's bin directory.

```
➤ cd "C:\Program Files\Java\jdk-17\bin"  
➤ javac C:\MyProjects\HelloWorld.java
```

With Path Set:

By adding C:\Program Files\Java\jdk-17\bin to the Path environment variable, you can run the javac and java commands from any directory.

```
➤ cd C:\MyProjects  
➤ javac HelloWorld.java  
➤ java HelloWorld
```

classpath:

The classpath is a parameter in the Java environment used by the Java compiler and JVM to locate class files and packages needed for running Java applications.

It tells Java where to look for user-defined classes and packages when executing or compiling a program. It can include directories, JAR files, and ZIP files.

For example, suppose there is an external library mylib.jar that Java program needs to use, located in the directory C:\MyProjects\lib.

Without Setting the Classpath:

You would need to specify the classpath each time you compile or run the program.

```
➤ cd C:\MyProjects  
➤ javac -cp lib/mylib.jar HelloWorld.java
```

```
➤ java -cp .;lib/mylib.jar HelloWorld
```

With Classpath Set:

You can set the classpath environment variable to include C:\MyProjects\lib\mylib.jar. This tells Java where to find the necessary classes and packages.

```
➤ set CLASSPATH=C:\MyProjects\lib\mylib.jar;.  
➤ cd C:\MyProjects  
➤ javac HelloWorld.java  
➤ java HelloWorld
```

java.lang Package and Its Classes:

The java.lang package is a fundamental package in Java SE that is automatically imported into every Java program. It contains essential classes for basic programming, such as basic data types, mathematical operations, string manipulation, and system operations.

Key Classes:

- **Object:** The root class of the Java class hierarchy. Every class in Java directly or indirectly inherits from this class.
- **String:** Represents sequences of characters and is immutable.
- **Math:** Provides methods for performing basic numeric operations like exponentiation, logarithms, and trigonometry.
- **System:** Contains methods and fields for standard input, output, and error, and also provides methods to interact with the environment.
- **Thread:** Represents a thread of execution in a program. It allows for concurrent execution of code.
- **Exception and Error:** Base classes for handling exceptions and errors in the Java runtime environment.

These classes are automatically imported into every Java program, so you don't need to explicitly import the java.lang package

Math class:

The Math class in Java is part of the java.lang package and provides methods for performing basic numeric operations such as elementary exponential, logarithm, square root, and trigonometric functions. Here are some of the key methods provided by the Math class:

1. abs(): Returns the absolute value of a number.

```
public static int abs(int a)
public static double abs(double a)
```

2. max(): Returns the greater of two values.

```
public static int max(int a, int b)
public static double max(double a, double b)
```

3. min(): Returns the smaller of two values.

```
public static int min(int a, int b)
public static double min(double a, double b)
```

4. sqrt(): Returns the correctly rounded positive square root of a double value.

```
public static double sqrt(double a)
```

5. pow(): Returns the value of the first argument raised to the power of the second argument.

```
public static double pow(double a, double b)
```

6. random(): Returns a double value with a positive sign, greater than or equal to 0.0 and less than 1.0.

```
public static double random()
```

```
public class MathExample {  
    public static void main(String[] args) {  
        double a = 9.0;  
        double b = 2.0;  
  
        System.out.println("Square root of " + a + " is: " + Math.sqrt(a));  
        System.out.println(a + " raised to the power of " + b + " is: " + Math.pow(a, b));  
        System.out.println("Absolute value of -5 is: " + Math.abs(-5));  
        System.out.println("Maximum of 3 and 7 is: " + Math.max(3, 7));  
        System.out.println("Random number between 0.0 and 1.0: " + Math.random());  
    }  
}
```

Output:

Square root of 9.0 is: 3.0
9.0 raised to the power of 2.0 is: 81.0
Absolute value of -5 is: 5
Maximum of 3 and 7 is: 7
Random number between 0.0 and 1.0: 0.7968459066553623

Wrapper Classes in Java:

Wrapper classes provide a way to use primitive data types (int, char, boolean, etc.) as objects. Each primitive data type has a corresponding wrapper class in the java.lang package.

- byte -> Byte
- short -> Short
- int -> Integer
- long -> Long
- float -> Float
- double -> Double
- char -> Character
- boolean -> Boolean

Wrapper classes are useful in scenarios where objects are required, such as in collections (e.g., ArrayList, HashMap), which cannot store primitive types directly.

Autoboxing and Unboxing

Autoboxing is the automatic conversion that the Java compiler makes between the primitive types and their corresponding object wrapper classes.

Autoboxing occurs when a primitive value is assigned to a variable of a wrapper class or passed as an argument to a method expecting an object.

For example, converting an int to an Integer.

```
public class AutoboxingExample {  
    public static void main(String[] args) {  
        int num = 10;  
        Integer obj = num; // Autoboxing  
        System.out.println("Integer object: " + obj);  
    }  
}
```

Output:

Integer object: 10

Unboxing is the reverse process, where the Java compiler automatically converts an object of a wrapper class to its corresponding primitive type.

Unboxing occurs when an object of a wrapper class is assigned to a variable of its corresponding primitive type or passed as an argument to a method expecting a primitive.

For example, converting an Integer to an int.

```
public class UnboxingExample {  
    public static void main(String[] args) {  
        Integer obj = Integer.valueOf(10);  
        int num = obj; // Unboxing  
        System.out.println("Primitive int: " + num);  
    }  
}
```

```
}  
}
```

Output:

Primitive int: 10

java.util package:

It Contains the collections framework, legacy collection classes, event model, date and time facilities, internationalization, and miscellaneous utility classes (a string tokenizer, a random-number generator, and a bit array).

Key Classes and Interfaces:

ArrayList, LinkedList, HashSet, HashMap: Implementation classes of common data structures like lists, sets, and maps.

Collections: Collections is a utility class in the java.util package that provides static methods for operating on collections.

Date and Calendar: Used for handling dates and times (note: now largely replaced by the java.time package in Java 8+).

Scanner: A simple text scanner used for parsing input.

Formatter: The Formatter class is designed to format data for output in a specified style

Comparator and Comparable: Interfaces for comparing objects.

Formatter class:

The Formatter class is a part of the java.util package which provides support for formatting strings, numbers, dates, and other objects. It is designed to mimic the functionality of printf in languages like C, offering a powerful way to create formatted output.

```
import java.util.Formatter;  
  
public class FormatterExample {  
    public static void main(String[] args) {  
        Formatter formatter = new Formatter();  
        formatter.format("Name: %s, Age: %d", "Pavan", 40);  
    }  
}
```

<pre> System.out.println(formatter); } } </pre>
Output: Name: Pavan, Age: 40

Random class:

The Random class is a part of the java.util package which is used to generate pseudorandom numbers of different data types like int, long, float, double, and boolean values.

<pre> import java.util.Random; public class RandomIntegerInRange { public static void main(String[] args) { int a = 10; // Lower bound int b = 50; // Upper bound Random random = new Random(); int randomInt = random.nextInt(b - a + 1) + a; System.out.println("Random Integer between " + a + " and " + b + ": " + randomInt); } } </pre>
Output: Random Integer between 10 and 50: 25

java.time Package:

The java.time package, introduced in Java 8, provides a comprehensive API for handling dates, times, instants, and durations. Here are some key classes and their uses

- **LocalDate:** Represents a date without a time-zone (e.g., 2024-09-29).
- **LocalTime:** Represents a time without a time-zone (e.g., 10:15:30).

- `LocalDateTime`: Combines date and time without a time-zone (e.g., 2024-09-29T10:15:30).
- `ZonedDateTime`: Combines date and time with a time-zone (e.g., 2024-09-29T10:15:30+05:30[Asia/Kolkata]).
- `Instant`: Represents a point in time (e.g., a timestamp).
- `Duration`: Represents a time-based amount of time (e.g., '34.5 seconds').
- `Period`: Represents a date-based amount of time (e.g., '2 years, 3 months, and 4 days').

```
import java.time.LocalDate;
import java.time.LocalDateTime;
import java.time.LocalTime;

public class TimeExample {
    public static void main(String[] args) {
        LocalTime currentTime = LocalTime.now();

        LocalDate currentDate = LocalDate.now();

        LocalDateTime currentDateTime = LocalDateTime.now();

        System.out.println("Current time : " + currentTime);
        System.out.println("Current Date: " + currentDate);
        System.out.println("Current current Date and time: " + currentDateTime);
    }
}
```

Output:

```
Current time : 01:47:11.506370114
Current Date: 2024-09-29
Current current Date and time: 2024-09-29T01:47:11.507282539
```

`java.time.Instant`:

The `java.time.Instant` class in Java represents a specific moment on the timeline, typically used to record event timestamps. It is part of the `java.time` package introduced in Java 8. Here are some key points about the `Instant` class:

Key Features:

- **Precision:** Represents time to nanosecond precision.
- **Epoch:** Measures time from the epoch of 1970-01-01T00:00:00Z.

```
//program to measure the execution time of a piece of code

import java.time.Duration;
import java.time.Instant;

public class ExecutionTimeExample {
    public static void main(String[] args) {
        // Record the start time
        Instant start = Instant.now();

        // Code block whose execution time is to be measured
        performTask();

        // Record the end time
        Instant end = Instant.now();

        // Calculate the duration between start and end
        Duration timeElapsed = Duration.between(start, end);
        System.out.println("Time taken: " + timeElapsed.toMillis() + "
milliseconds");
    }

    static void performTask() {
        // Simulate a task by making the thread sleep for 2 seconds
        try {
            Thread.sleep(2000);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}
```

Output:

Time taken: 2000 milliseconds

Formatting date and time:

Formatting date and time in Java can be efficiently handled using the **DateTimeFormatter class**, which was introduced in Java 8. This class allows to define custom patterns for formatting and parsing date-time objects (converting a string representation of a date and/or time into a java.time object).

```
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

public class BasicFormattingExample {
    public static void main(String[] args) {
        LocalDateTime now = LocalDateTime.now();
        DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd-MM-
yyyy HH:mm:ss");
        String formattedDateTime = now.format(formatter);
        System.out.println("Local DateTime: " + now);
        System.out.println("Formatted DateTime: " + formattedDateTime);
    }
}
```

Output:

```
Local DateTime: 2024-09-29T02:16:39.519763262
Formatted DateTime: 29-09-2024 02:16:39
```

```
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.time.format.FormatStyle;

public class LocalizedFormattingExample {
    public static void main(String[] args) {
        LocalDate date = LocalDate.now();
        DateTimeFormatter formatter =
DateTimeFormatter.ofLocalizedDate(FormatStyle.FULL);
        String localizedFormattedDate = date.format(formatter);
        System.out.println("Localized Formatted Date: " +
localizedFormattedDate);
    }
}
```

Output:

Localized Formatted Date: Sunday, September 29, 2024

Temporal Adjusters Class:

The TemporalAdjusters class in Java, part of the java.time.temporal package, provides a set of common and useful TemporalAdjuster implementations. These adjusters are used to manipulate temporal objects, such as dates and times, in a flexible and reusable way

It Includes methods to adjust dates to the first or last day of the month, the first or last day of the year, the next or previous occurrence of a specific day of the week, and more.

```
import java.time.LocalDate;
import java.time.DayOfWeek;
import java.time.temporal.TemporalAdjusters;

public class TemporalAdjustersExample {
    public static void main(String[] args) {
        LocalDate today = LocalDate.now();

        // First day of the current month
        LocalDate firstDayOfMonth = today.with(TemporalAdjusters.firstDayOfMonth());
        System.out.println("First day of this month: " + firstDayOfMonth);

        // Last day of the current month
        LocalDate lastDayOfMonth = today.with(TemporalAdjusters.lastDayOfMonth());
        System.out.println("Last day of this month: " + lastDayOfMonth);

        // First day of the next month
        LocalDate firstDayOfNextMonth = today.with(TemporalAdjusters.firstDayOfNextMonth());
        System.out.println("First day of next month: " + firstDayOfNextMonth);

        // First Monday of the current month
        LocalDate firstMonday = today.with(TemporalAdjusters.firstInMonth(DayOfWeek.MONDAY));
        System.out.println("First Monday of this month: " + firstMonday);

        // Next Friday
        LocalDate nextFriday = today.with(TemporalAdjusters.next(DayOfWeek.FRIDAY));
        System.out.println("Next Friday: " + nextFriday);

        // Previous Friday
        LocalDate previousFriday = today.with(TemporalAdjusters.previous(DayOfWeek.FRIDAY));
        System.out.println("Previous Friday: " + previousFriday);
    }
}
```

Output:

First day of this month: 2024-09-01

Last day of this month: 2024-09-30
First day of next month: 2024-10-01
First Monday of this month: 2024-09-02
Next Friday: 2024-10-04
Previous Friday: 2024-09-27

Exception Handling

Exceptions are exceptional/unusual/abnormal events that occur during the execution of programs that disrupts the normal flow of instructions

If an Exception occurs, the code should handle it otherwise program will terminate abnormally. These abnormal terminations may crash local operating systems, may provide hanged out situations to the network etc.

The code that specifies what to do in specific exception scenarios so that the execution of the business code can continue uninterrupted is called exception handling.

Exception handling ensures that the flow of the program doesn't break when an exception occurs.

Ex:

1. ArithmeticException

Occurs when an exceptional arithmetic condition has occurred, such as division by zero.

```
public class Main {  
    public static void main(String[] args) {  
        int d=0;  
        int result=100/d;    //dividing by zero  
        System.out.println(result);  
    }  
}
```

2.ArrayIndexOutOfBoundsException

Occurs when trying to access an array with an illegal index.

```
public class Main {  
    public static void main(String[] args) {  
        int[] a = new int[5];  
        System.out.println(a[100]);  
        //trying to access array element with index 100 which has only five  
        elements  
    }  
}
```

3. NullPointerException

Occurs when trying to use an object reference that has the null value.

```
public class Main {  
    public static void main(String[] args) {  
        String str= null;  
        int len=str.length(); //trying to access the object reference which has null  
        System.out.println(len);  
    }  
}
```

4.NumberFormatException

Occurs when trying to convert a string into a numeric format that is not appropriate.

```
public class Main {  
    public static void main(String[] args) {  
        String str= "ByteXL";  
        int i = Integer.parseInt(str); //trying to convert String to int  
        System.out.println(i);  
    }  
}
```

5. FileNotFoundException

Occurs when trying to access a file that does not exist.

Exception Handling keywords:

Five keywords are used for exception handling. They are try, catch, throw, throws and finally.

we place a piece of vulnerable code (i.e., which might cause some exception) in a block called try block

When an exception taken place in the try block, the program execution will be stopped automatically and the control will be transferred to catch block or finally block by skipping the rest of the statements in the try block.

After executing the appropriate catch block, the control never come back to the try block even we use return statement in catch block.

It is always recommended to write a separate catch block for each and every runtime problem.

try

The try block contains code that might throw an exception. If an exception occurs, it is thrown to the corresponding catch block.

Java try block must be followed by either catch or finally block. No intermediate statements are allowed between try and catch block.

```
try {  
    block of statements that causes exception and are to be  
    monitored by JVM  
} catch(ExceptionclassName ref){.....}
```

(or)

```
try {  
    block of statements that causes exception and are to be  
    monitored by JVM  
} finally{.....}
```

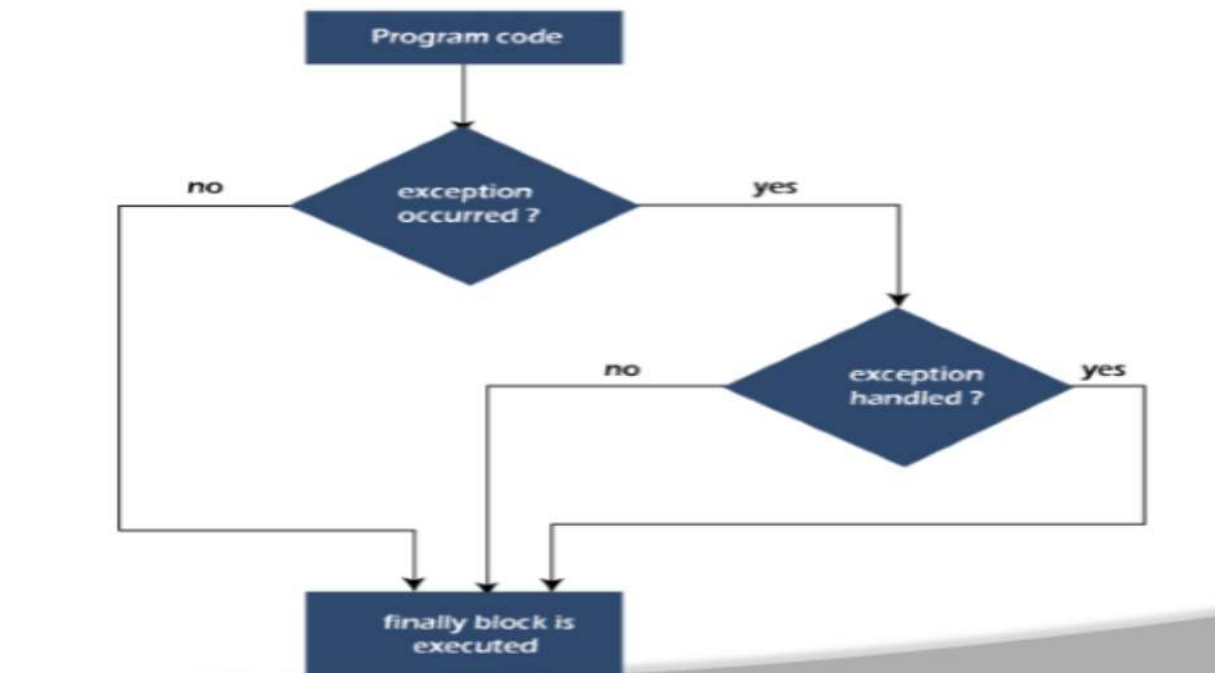
catch

The catch block handles the exception thrown by the try block by declaring the type of exception within the parameter. Multiple catch blocks can be used to handle different types of exceptions with a single try block.

```
try {  
    block of statements that causes exception and are to be  
    monitored by JVM  
} catch(ExceptionclassName ref){.....}
```

finally

Java finally block is always executed whether an exception is handled or not. It is typically used for resource cleanup such as closing the database connection, close the file etc.



throw

The throw keyword is used to explicitly throw an exception.

throws

The throws keyword is used in a method declaration to declare that the method might throw one or more exceptions.

Exception Handling Mechanism:

When an exception occurs during the execution of the code in the try block, JVM creates an exception object and declares that something wrong has happened.

The exception object contains a lot of useful debugging information such as name of thread, file name, line of code at which exception occurred, name of exception and it's description etc.

When an exception occurs within a try block, the Java runtime system searches for a matching catch block. If a matching catch block is found, the exception is

handled. If no matching catch block is found, the exception is propagated up the call stack, and the program terminates.

```
import java.util.Scanner;

public class ExceptionHandlingExample
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);

        try
        {
            // Code that might throw an exception
            System.out.print("Enter a number: ");
            int number = sc.nextInt();
            int result = 10 / number;
            System.out.println("Result: " + result);
        }
        catch (ArithmeticException e)
        {
            // Catch block to handle ArithmeticException
            System.out.println(e);
        }
        finally
        {
            // Code that will always execute, regardless of exceptions
            sc.close();
            System.out.println("Finally block executed.");
        }
    }
}
```

If this program is executed, it no longer terminates abruptly whatever be the numeric value of the number. If the number is given as zero then the output of the above program is given as follows

```
Enter a number: 0
java.lang.ArithmeticException: / by zero
Finally block executed.
```

Multiple Catch blocks:

The catch block must be used after the try block only. Multiple catch blocks can be used to handle different types of exceptions with a single try block. This usually happens if try block throws more than one type of exceptions and we want to handle them differently.

```
try {
    //code that throws exception
}
catch(Exception1 e) {
    //handle this way
}
catch(Exception2 e) {
    //handle that way
}
...
```

Ex:

```
import java.util.Scanner;
import java.util.InputMismatchException;

public class ExceptionHandlingExample
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);

        try
        {
            // Code that might throw an exception
            System.out.print("Enter a number: ");
            int number = sc.nextInt();
            int result = 10 / number;
            System.out.println("Result: " + result);
        }
        catch (ArithmeticException e)
        {
            // Catch block to handle ArithmeticException
            System.out.println(e);
        }
    }
}
```

```

    }
    catch (InputMismatchException e) {
        // Catch block to handle InputMismatchException
        System.out.println(e);
    }

    finally
    {
        // Code that will always execute, regardless of exceptions
        sc.close();
        System.out.println("Finally block executed.");
    }
}

```

Output:

In this program, when number value is given wrongly as String, then InputMismatchException occurs and it is handled in the second catch block. In this case, the output of the program is

```

Enter a number: pavan
java.util.InputMismatchException
Finally block executed.

```

If multiple catch blocks have similar code, in Java SE 7 and later, we can combine multiple catch blocks into a single one using pipe (|) operator.

For ex, the following catch block can handle ArithmeticException as well as InputMismatchException

```

catch (ArithmeticException | InputMismatchException e)
{
    System.out.println(e);
}

```

When multiple catch blocks are there, Order of catch blocks does matter.

Consider the following catch blocks:

```
catch(Exception e) {}
```

```
catch(ArrayIndexOutOfBoundsException e) {}
```

Since, Exception is the superclass of all the exceptions, all the exceptions (including ArrayIndexOutOfBoundsException) will be caught by the first catch block. So, second catch block is unreachable and results in a compilation error

In general, **catch blocks must be arranged from most specific to most general.** So, the following catch blocks are valid:

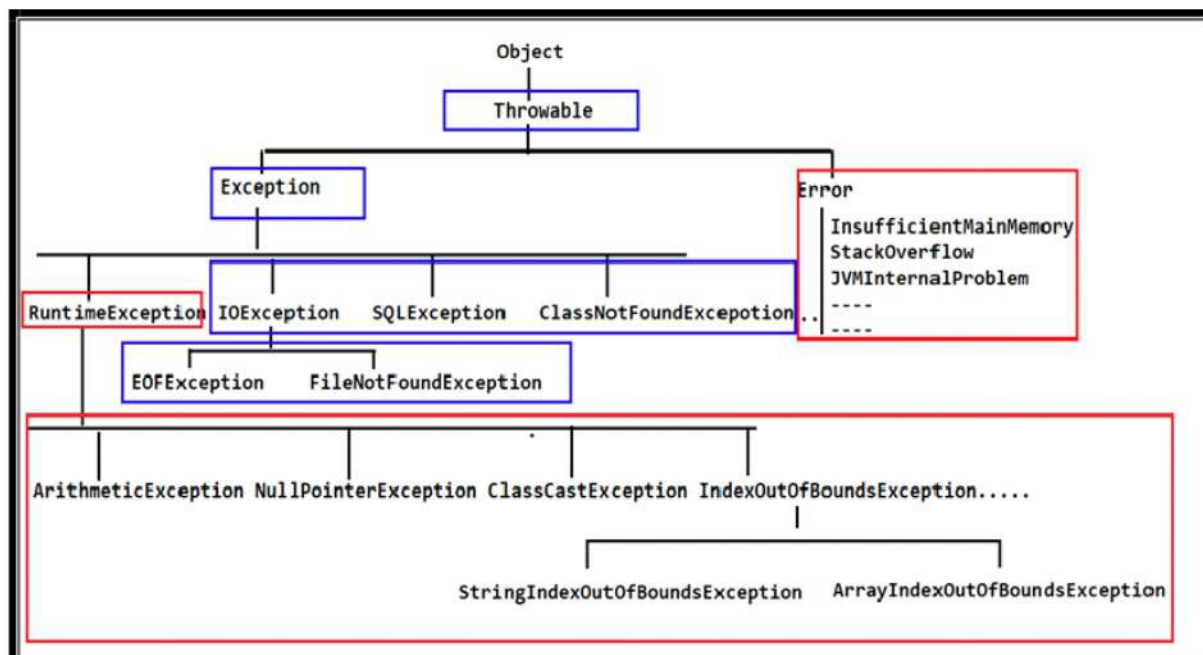
```
catch(ArrayIndexOutOfBoundsException e) {}
```

```
catch(Exception e) {}
```

Here, the first catch block only catches ArrayIndexOutOfBoundsException whereas second one catches everything else.

Hierarchy of Standard Exception Classes

Java provides a hierarchy of exception classes that represent various exceptional situations



The root of this exception class hierarchy is the Throwable class. It is part of the java.lang package. Conventionally, two subclasses of this Throwable class, namely Exception and Error are provided to indicate exceptional situations

The class Error represents serious problematic scenarios (we shall call them errors) that applications can neither anticipate nor recover from. Examples of such errors are out of memory error, stack overflow error, class format error, no class definition found error etc.

Exceptions are exceptional/unusual/abnormal events that occur during the execution of programs that disrupts the normal flow of instructions

Errors	Exceptions
Errors are usually raised by the environment in which the application is running. For example, an error will occur due to a lack of system resources.	Exceptions are caused by the code of the application itself.
It is not possible to recover from an error.	The use of try-catch blocks can handle exceptions and recover the application from them.
Errors occur at run-time and are not known by the compiler; hence, they are classified as “unchecked.”	Exceptions can be “checked” or “unchecked,” meaning they may or may not be caught by the compiler.
“OutOfMemory” and “StackOverflow” are examples of errors.	“IndexOutOfBoundsException” is an example of an unchecked exception, while “ClassNotFound” is an example of a checked exception.

Throwable class:

The root of this exception class hierarchy is the Throwable class. It is part of the java.lang package. Conventionally, two subclasses of this Throwable class, namely Exception and Error are provided to indicate exceptional situations.

Key methods:

- getMessage(): Returns the detailed message of the throwable.
- getCause(): Returns the cause of the throwable.
- printStackTrace(): Prints the throwable and its backtrace to the standard error stream.
- getStackTrace(): Provides programmatic access to the stack trace information

checked Exceptions:

Checked Exception is an exception recognized at compilation time, but, not occurred at compilation time.

Checked exceptions are those that must be caught or declared to be thrown.

Checked exceptions should be handled in the code to avoid compile time errors.

Java considers Throwable and any of its subclass that is not also a subclass of either RuntimeException or Error as checked exceptions.

Unchecked Exceptions:

unchecked Exceptions are not recognized at compilation time, these exceptions are recognized at runtime by JVM

Unchecked exceptions (also called runtime exceptions) are those which need not be caught or declared to be thrown explicitly.

RuntimeException and its sub classes and Error and its subclasses are the examples for Unchecked Exceptions

throws keyword:

The throws keyword is used in method declarations to indicate that the method may throw a specific type of exception.

This allows the caller of the method to be aware of potential exceptions and handle them appropriately.

For example, consider the following method

```
public void doSome() throws IOException {  
    // Method body  
}
```

In this example:

- doSome() method declares that it may throw an IOException.
- The caller of this method must either:

- Handle the IOException using a try-catch block.
- Declare that the method itself may throw an IOException using the throws keyword.

we can use throws keyword to delegate responsibility of exception handling to the caller(jvm or another method)

throw keyword:

The throw keyword is used to explicitly throw an exception object. The throw keyword is used within a method or a block of code to throw an exception.

It can be used to throw both checked and unchecked exceptions but Commonly used to throw custom exceptions.

Syntax:

```
throw new ExceptionType(message);
```

This allows you to signal that an error condition has occurred and to provide information about the error to the caller.

For example, consider the following code

```
public void divide(int a, int b) {  
    if (b == 0) {  
        throw new IllegalArgumentException("Cannot divide by zero.");  
    }  
    // ... perform division ...  
}
```

In the above example,

The divide() method checks if the divisor b is zero. If it is, an IllegalArgumentException is thrown with a descriptive message.

The caller of this method must handle the exception using a try-catch block or declare that the method itself may throw the exception.

Custom Exceptions:

custom exceptions are user-defined exceptions that extend the Exception class or one of its subclasses.

Custom exceptions would be defined by the developers as per their application requirements.

Creating a Custom Exception

To create a custom exception, you need to:

1. **Extend the Exception class:** Create a new class that extends the Exception class or one of its subclasses like RuntimeException.
2. **Provide a constructor:** provide at least one constructor in the newly created class that takes an optional message as a parameter. This message can be used to provide more details about the exception.

```
public class MyCustomException extends Exception {  
    public MyCustomException(String message) {  
        super(message);  
    }  
}
```

Ex:

Write a Java program that prompts the user to enter their age. If the entered age is negative or greater than 120, the program should throw a custom exception called AgeException with an appropriate error message. Otherwise, it should print the entered age. Ensure that the program handles the custom exception and displays the error message to the user.

```
//program to illustrate custom Exception  
import java.util.Scanner;  
  
class AgeException extends Exception  
{  
    public AgeException(String message)  
    {  
        super(message);  
    }  
}
```

```

public class CustomExceptionExample
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);

        try
        {
            System.out.print("Enter your age: ");
            int age = sc.nextInt();

            if (age < 0) {
                throw new AgeException("Age cannot be negative.");
            } else if (age > 120) {
                throw new AgeException("Age is too high.");
            }

            System.out.println("Your age is: " + age);
        } catch (AgeException e) {
            System.out.println("Error: " + e.getMessage());
        } finally {
            sc.close();
        }
    }
}

```

Output1:

Enter your age: 44
Your age is: 44

Output2:

Enter your age: -25
Error: Age cannot be negative.

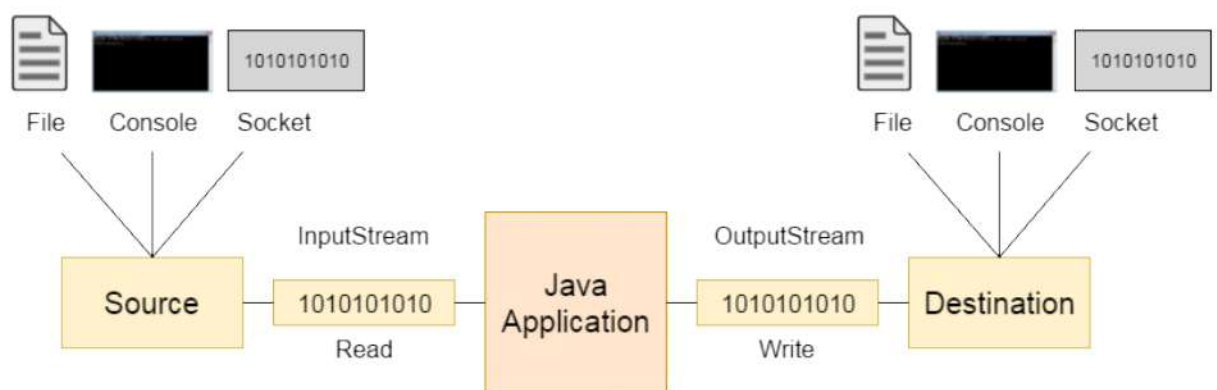
Output3:

Enter your age: 130
Error: Age is too high.

I/O Streams:

Java programs generally takes input data , process on the data by performing operations and produces the output. To perform I/O operations, we must understand the concept of I/O Streams in java.

- In Java, streams are the sequence of data that are read from the source and written to the destination.
- there are two types of streams: input streams and output streams
- An input stream is used to read data from the source. Java application uses an input stream to read data from a source; it may be a file, an array, peripheral device or socket.
- And, an output stream is used to write data to the destination. Java application uses an output stream to write data to a destination; it may be a file, an array, peripheral device or socket.



Depending upon the data a stream holds, streams can be classified as

- a) Byte Streams
- b) Character Streams

Byte Streams

Byte Streams represent data in the form of Bytes. Byte Stream is used to read and write a single byte. Byte Streams are used to handle any text, images, audio and video files.

Ex: FileInputStream

FileOutputStream

BufferedInputStream

BufferedOutputStream

Character streams

The Java platform stores character values using Unicode conventions. Character Streams represent data in the form of characters of 2 bytes i.e., 16 bit unicode

Ex: FileReader

FileWriter

BufferedReader

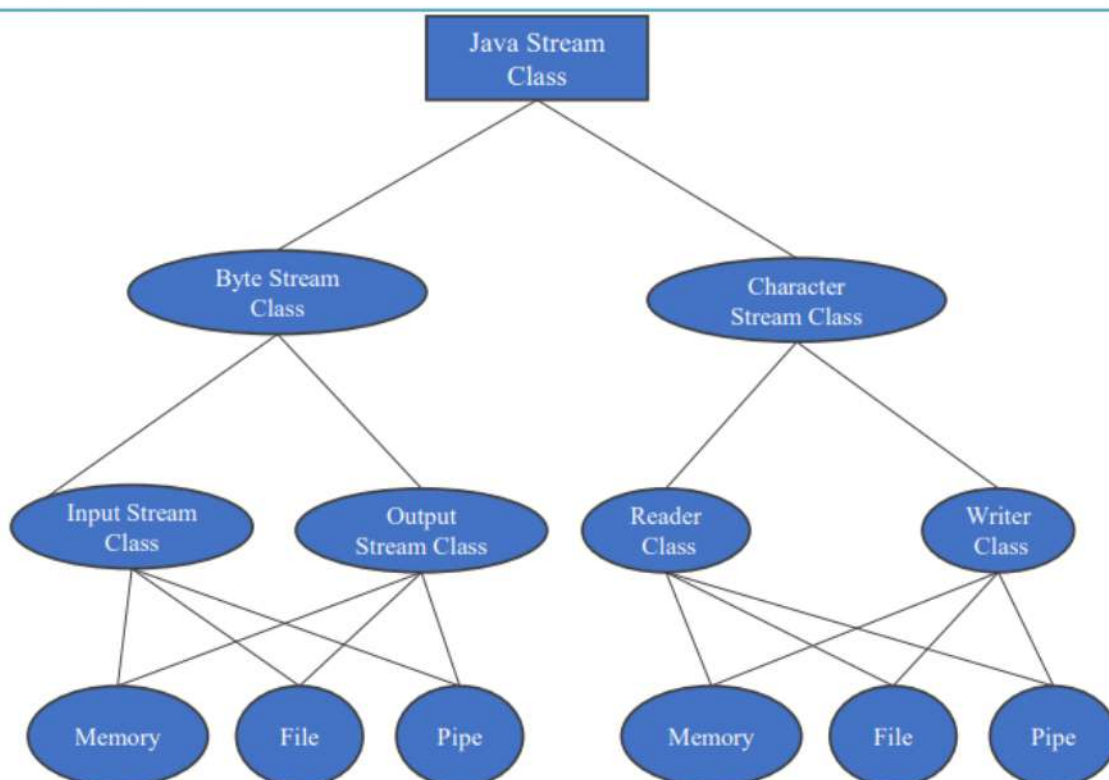
BufferedOutputStream

Stream Classes:

The java.io package contains large number of stream classes that provide capabilities for processing all types of data. These classes can be categorized into ByteStream classes and CharacterStream classes.

Byte stream classes provide support for handling I/O operations on bytes where as character stream classes provide support for handling I/O operations on characters.

These two groups may further classified based on their purpose which is shown in following figure.



Byte Stream classes:

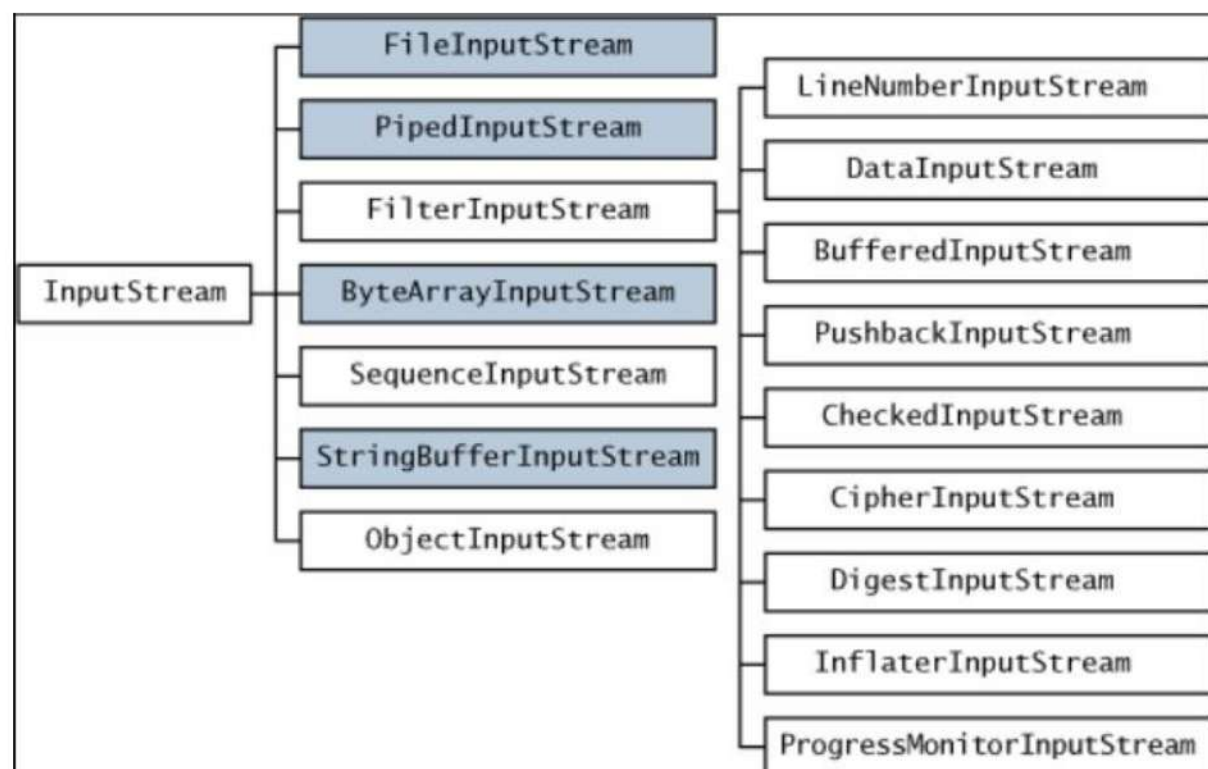
Byte stream classes in Java are fundamental for handling binary data, and they all inherit from the `InputStream` and `OutputStream` classes.

InputStream class:

This abstract class is the superclass of all classes representing an input stream of bytes. Since it is abstract we can not create instances of this class, we must use subclasses that inherit from this class.

Hierarchy:

The following figure shows the class hierarchy of `InputStream` class.



InputStream inherits from the Object class; other classes inherit directly from InputStream. One of InputStream's descendants, FilterInputStream, is itself an abstract class with four children.

Methods of InputStream Class:

Method	Description
<code>read()</code>	Read a byte from the input stream
<code>read(byte b[])</code>	Read an array of bytes into b
<code>read(byte b[], int n, int m)</code>	Reads m bytes into b starting from n th byte
<code>available()</code>	Gives number of bytes available in the input
<code>skip(n)</code>	Skips over n bytes from the input stream
<code>reset()</code>	Goes back to the beginning of the stream
<code>close()</code>	Close the input steam

Note:

Note that the class **DataInputStream** extends **FilterInputStream** and implements the interface **DataInput**. Therefore, the **DataInputStream** class implements the methods described in **DataInput** in addition to using the methods of **InputStream** class. The **DataInput** interface contains the following methods:

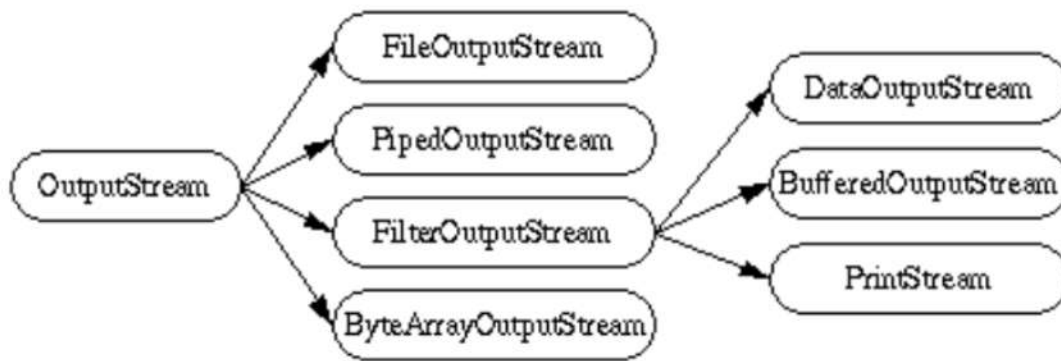
• <code>readShort()</code>	• <code>readDouble()</code>
• <code>readInt()</code>	• <code>readLine()</code>
• <code>readLong()</code>	• <code>readChar()</code>
• <code>readFloat()</code>	• <code>readBoolean()</code>
• <code>readUTF()</code>	

OutputStream class:

This abstract class is the superclass of all classes representing an output stream of bytes. Since it is abstract we can not create instances of this class. we must use subclasses that inherit from this class.

Hierarchy:

The following figure shows the class hierarchy of OutputStream class.



OutputStream inherits from the Object class; four classes inherit directly from OutputStream. One of OutputStream's descendants, FilterOutputStream, is itself an abstract class with three descendants.

Methods of OutputStream Class:

Method	Description
write ()	Write a byte from the input stream
write (byte b[])	Write all bytes in the array b to the output steam
write (byte b[], int n, int m)	Write m bytes from array b starting from n th byte
close ()	Close the output stream
flush ()	Flushes the output stream

The following is an overview of the input and output stream classes that inherit directly from InputStream and OutputStream that are not themselves abstract classes.

FileInputStream and FileOutputStream

Used to read data from or write data to a file on the native file system.

PipedInputStream and PipedOutputStream

Implements the input and output components of a pipe. Pipes are used to channel the output from one program (or thread or code block) into the input of another. A PipedInputStream must be connected to a PipedOutputStream and vice versa.

ByteArrayInputStream and ByteArrayOutputStream

Reads data from or writes data to a byte array in memory.

SequenceInputStream

Concatenates multiple input streams into one input stream.

StringBufferInputStream

Allows programs to read from a StringBuffer as if it were an input stream.

Filtered Streams

FilterInputStream and FilterOutputStream are subclasses of InputStream and OutputStream, respectively, and are both themselves abstract classes. These classes define the interface for filtered streams. Filtered streams process the data as it is being read or written. For example, the BufferedInputStream and BufferedOutputStream classes buffer the data while reading and writing to speed it up.

DataInputStream and DataOutputStream

Reads or writes primitive Java data types in a machine independent format.

BufferedInputStream and BufferedOutputStream

This is an efficient stream that buffers data while reading or writing.

LineNumberInputStream

An input stream that keeps track of line numbers while reading.

PushbackInputStream

An input stream with a one-byte pushback buffer.

PrintStream

An output stream with convenient printing methods.

FileInputStream:

```
//program that illustrate reading of data from file using FileInputStream
import java.io.FileInputStream;
public class FileInput1
{
    public static void main(String args[])
    {
        try
        {
            FileInputStream fin=new FileInputStream("./data.txt");
            int i=0;
            while((i=fin.read())!=-1)
            {
                System.out.print((char)i);
            }
            fin.close();
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }
}
```

FileOutputStream:

```
/*      program takes the given string and writes it to a file named testout.txt      using
FileOutputStream class */
import java.io.FileOutputStream;
public class FileOutput1
{
    public static void main(String args[])
    {
        try
        {
            FileOutputStream fout=new FileOutputStream("./testout.txt");
            String s="Welcome to bitlabs java learning.";
            byte b[]=s.getBytes();//converting string into byte array
            fout.write(b);
            fout.close();
            System.out.println("success...");
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }
}
```

```

//program that reads data from keyboard and write to a file
import java.io.*;
public class FileOutput2
{
    public static void main(String args[])
    {
        try
        {

            DataInputStream dis = new DataInputStream(System.in);
            FileOutputStream fout=new FileOutputStream("./testout2.txt",true);
            System.out.println("enter some text (@at the end):");
            char ch;

            //read character into ch using dis then write into file using fout
            //repeat this procedure as long as read character is not equal to @
            while((ch=(char)dis.read())!='@')
            {
                fout.write(ch);
            }

            fout.close();
            System.out.println("success...");
        }
        catch(Exception e)
        {
            System.out.println(e);
        }
    }
}

```

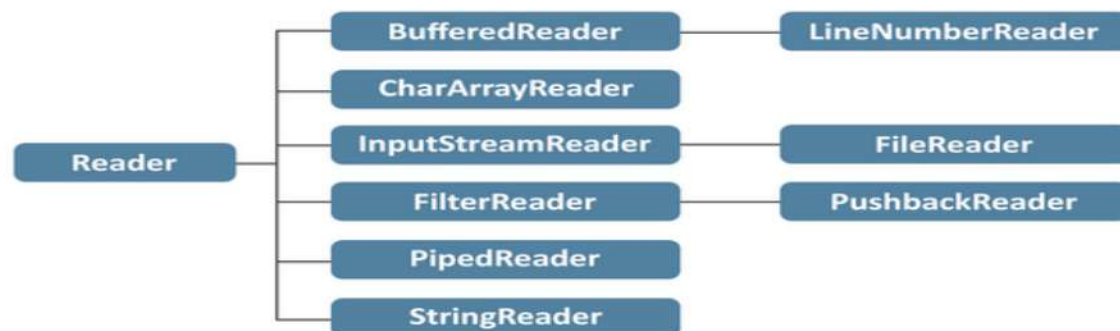
```
//note: if you want to append data then use following statement
// FileOutputStream fout=new FileOutputStream("./testout2.txt", true);
```

Reader and writer:

InputStream and OutputStream read/write one byte at a time. When you're dealing with text in Java, you should use String and char, instead of working directly with bytes. So, instead of using InputStream and OutputStream, use [java.io.Reader](#) and [java.io.Writer](#). These are very similar to InputStream and OutputStream, except that now [read\(\)](#) and [write\(int b\)](#) handle char values (2 bytes, 0 to $2^{16}-1$) instead of byte values (0 to 2^8-1).

Reader class:

A reader is used when a user wants to read character-based data from a source of the data. The Java IO Reader class is an abstract class that is used for expressing character streams.

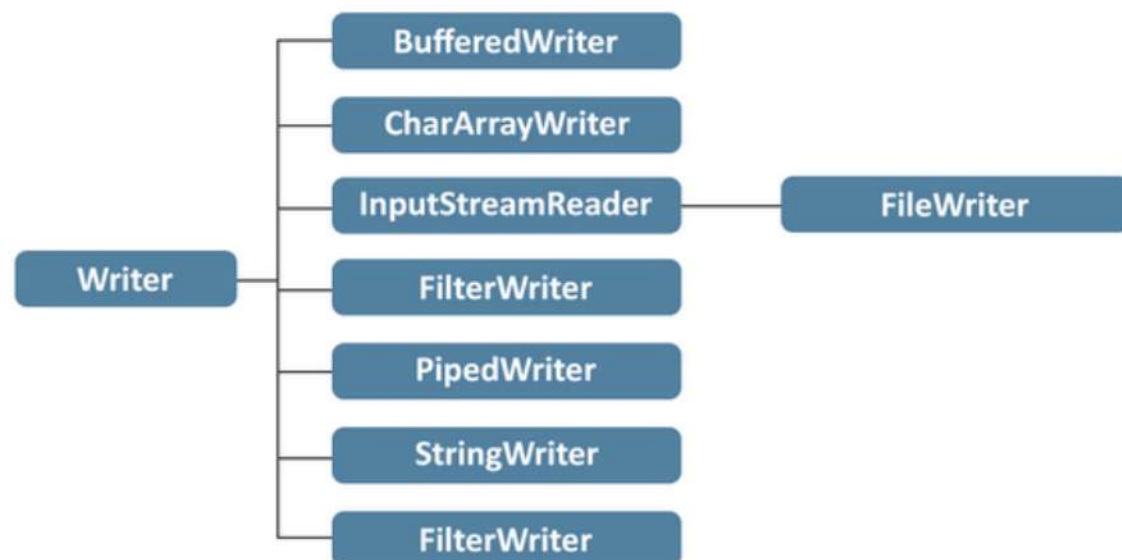


Reader class methods

Modifier and Type	Method and Description
abstract void	close() Closes the stream and releases any system resources associated with it.
void	mark(int readAheadLimit) Marks the present position in the stream.
boolean	markSupported() Tells whether this stream supports the mark() operation.
int	read() Reads a single character.
int	read(char[] cbuf) Reads characters into an array.
abstract int	read(char[] cbuf, int off, int len) Reads characters into a portion of an array.
int	read(CharBuffer target) Attempts to read characters into the specified character buffer.
boolean	ready() Tells whether this stream is ready to be read.
void	reset() Resets the stream.
long	skip(long n) Skips characters.

Writer class:

Writer is an abstract class utilized for writing to the character streams. The only methods in Java IO Writer that a subclass must implement are `write(char[], int, int)`, `flush()`, and `close()`. Most subclasses, however, will override several methods explained here to implement higher efficiency, additional functionality, or both.



Writer class methods:

Modifier and Type	Method and Description
Writer	<code>append(char c)</code> Appends the specified character to this writer.
Writer	<code>append(CharSequence csq)</code> Appends the specified character sequence to this writer.
Writer	<code>append(CharSequence csq, int start, int end)</code> Appends a subsequence of the specified character sequence to this writer.
abstract void	<code>close()</code> Closes the stream, flushing it first.
abstract void	<code>flush()</code> Flushes the stream.
void	<code>write(char[] cbuf)</code> Writes an array of characters.
abstract void	<code>write(char[] cbuf, int off, int len)</code> Writes a portion of an array of characters.
void	<code>write(int c)</code> Writes a single character.
void	<code>write(String str)</code> Writes a string.
void	<code>write(String str, int off, int len)</code> Writes a portion of a string.

When to use InputStream versus Reader

The decision criterion for InputStream versus Reader or OutputStream versus Writer is pretty simple: if you're dealing with text, use Reader or Writer. If you're dealing with inherently binary data (like a JPEG image, an MP4 video, executable code, a ZIP file), then use InputStream or OutputStream.

Reading From Keyboard

We can use standard streams to read data from keyboard

Reading Character:

To read characters, we can use standard character stream Reader.

The static System.in field corresponds to keyboard input. Unfortunately, it is a byte stream.

However, we can easily convert it to a character stream using InputStreamReader.

The following program loops to accept characters from the keyboard until the character '@' is entered.

```
import java.io.*;

public class ReadFromKeyboard
{
    public static void main(String args[]) throws IOException
    {
        InputStreamReader cin = new InputStreamReader(System.in);
        System.out.println("Enter characters, '@' to quit.");
        char c;
        do
        {
            c = (char) cin.read();
            System.out.print(c);
        } while(c != '@');
        cin.close();
    }
}
```

Reading String:

The following program uses the buffered character stream `BufferedReader` to read the string

Reads text from a character-input stream, buffering characters so as to provide for the efficient reading of characters, arrays, and lines. The buffer size may be specified, or the default size may be used. The default is large enough for most purposes.

In general, each read request made by a `Reader` causes a corresponding read request to be made of the underlying character or byte stream. It is therefore advisable to wrap a `BufferedReader` around any `Reader` whose `read()` operations may be costly, such as `FileReaders` and `InputStreamReaders`.

```
/*program to
read input from the keyboard using BufferedReader and write it to the file using Buffered
Writer */

import java.io.*;

public class FileOutput2 {
    public static void main(String args[]) {
        try {
            BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
            BufferedWriter bw = new BufferedWriter(new FileWriter("./testout2.txt", true));
            System.out.println("Enter some text (@ at the end):");
            char ch;
            // Read character into ch using reader then write into file using writer
            // Repeat this procedure as long as read character is not equal to @
            while ((ch = (char) br.read()) != '@') {
                bw.write(ch);
            }
            bw.close();
            System.out.println("Success...");
        } catch (Exception e) {
            System.out.println(e);
        }
    }
}
```

The code will read input from the keyboard using `BufferedReader` and write it to the file using `BufferedWriter`. The loop continues until you type "@".

Write a java program to copy contents of one file to another file

```
//program to copy contents of one file to another file
import java.io.*;

public class FileCopy {
    public static void main(String[] args) {
        try {
            BufferedReader br = new BufferedReader(new FileReader("input.txt"));
            BufferedWriter bw = new BufferedWriter(new FileWriter("output.txt"));
            String line;
            while ((line = br.readLine()) != null) {
                bw.write(line);
                bw.newLine();
            }
            br.close();
            bw.close();
            System.out.println("File copied successfully.");
        } catch (IOException e) {
            System.out.println("An error occurred: " + e.getMessage());
        }
    }
}
```